

**Міністерство освіти і науки України
Національний технічний університет України “Київський
політехнічний інститут імені Ігоря Сікорського”
Факультет інформатики та обчислювальної техніки Кафедра
інформаційних систем та технологій**

**Лабораторна робота №7
ШАБЛони «MEDIATOR», «FACADE», «BRIDGE»,
«TEMPLATE METHOD»
Варіант: 25. Installer generator**

Виконав:

Студент групи ІА-22

Птачик Р.С.

Перевірив:

Мягкий М.Ю.

Київ 2024

Зміст:

Тема:.....	3
Короткі теоретичні відомості.....	3
Реалізувати не менше 3-х класів відповідно до обраної теми	8
Реалізувати один з розглянутих шаблонів за обраною темою	9
Діаграма класів для шаблону bridge	13
Проблема, яку допоміг вирішити шаблон bridge	13
Переваги використання шаблону bridge.....	14
Код та висновок	15

Тема: шаблони «mediator», «facade», «bridge», «template method».

Мета: ознайомитися з шаблонами проектування «mediator», «facade», «bridge», «template method». Реалізувати частину функціоналу програми за допомогою одного з розглянутих шаблонів для досягнення конкретних функціональних можливостей та забезпечення ефективної взаємодії між класами.

Хід роботи:

Тема:

25. Installer generator (iterator, builder, factory method, bridge, interpreter, client-server)

Генератор інсталяційних пакетів повинен мати якийсь спосіб налаштування файлів, що входять в установку, установки вікон з інтерактивними можливостями (галочка - створити ярлик на робочому столі; ввести в текстове поле деякі дані, наприклад, ліцензійний ключ і т.д.). Генератор повинен вивести один файл .exe або .msi.

Короткі теоретичні відомості

Основні принципи проектування

1. Принцип Don't Repeat Yourself (DRY):

Уникайте повторень в коді. Подібні повторення можуть проявлятися у вигляді однакових блоків коду або дублюючої функціональності в різних частинах програми. Їх присутність може викликати проблеми:

- Погіршується читабельність коду.
- Помилки, знайдені в одному місці, можуть залишитися не виправленими в інших.
- Копіювання коду дублює не тільки функціональність, але й помилки. Для вирішення використовуйте рефакторинг: виділення загальних методів, інтерфейсів або класів.

2. Принцип Keep It Simple, Stupid! (KISS):

Прагніть до максимальної простоти в розробці. Система, що складається з дрібних і простих компонентів, є більш надійною, ніж складна структура. Прості рішення легше тестувати, підтримувати і розширювати. Філософія Unix також слідує цьому принципу: кожна програма виконує лише одну функцію, але виконує її якісно.

3. Принцип You Only Load It Once! (YOLO)

Ініціалізація змінних та конфігурацій повинна відбуватися одноразово на початку роботи програми. Це запобігає повторним зчитуванням даних з диска, які можуть значно вплинути на продуктивність.

4. Принцип Парето (80/20)

Більшість результатів зазвичай досягається за рахунок мінімальних зусиль:

- 80% навантаження на сервер створюють 20% функцій.
- 80% помилок можна виправити, вирішивши 20% ключових проблем. Орієнтуйтеся на ці 20% для максимального ефекту. Але пам'ятайте, що досягнення ідеалу (залишкових 20% результатів) потребує надмірних витрат.

5. Принцип You Ain't Gonna Need It (YAGNI)

Уникайте додавання зайвого функціоналу, який не потрібен тут і зараз. Загальні чи універсальні рішення, які не мають практичного застосування, не тільки ускладнюють код, але й уповільнюють розробку. Фокусуйтеся на реалізації ключових функцій, необхідних користувачам.

Шаблон «Mediator» (Посередник):

Призначення:

Посередник забезпечує взаємодію між об'єктами через окремий об'єкт, уникаючи прямого зв'язку між ними. Це зменшує залежності між компонентами, робить код більш гнучким і дозволяє повторно використовувати компоненти в різних контекстах.

Проблема:

При взаємодії між елементами, такими як текстові поля, кнопки чи чекбокси, складна логіка ускладнює їх повторне використання в інших сценаріях.

Рішення:

Посередник координує обмін інформацією між компонентами, знижуючи їхню залежність один від одного.

Приклад:

Пілоти літаків не спілкуються напряму, а взаємодіють через диспетчера, який організовує їхні дії.

Переваги:

- Зменшує залежності між компонентами.

- Централізує управління і спрощує взаємодію.

Недоліки:

- Може призводити до надмірної складності самого посередника.

Шаблон «Facade» (Фасад):**Призначення:**

Фасад створює єдиний інтерфейс доступу до складної підсистеми, приховуючи її внутрішню реалізацію. Це спрощує використання підсистеми та забезпечує зручність роботи з нею.

Проблема:

Робота з великою кількістю об'єктів складних бібліотек чи фреймворків може ускладнювати розробку і супровід коду через переплетення бізнес-логіки з деталями реалізації.

Рішення:

Фасад надає простий інтерфейс, що приховує зайві деталі, залишаючи лише необхідну функціональність.

Приклад:

Оператор служби підтримки магазину виступає фасадом, допомагаючи клієнту взаємодіяти із замовленням, оплатою та доставкою.

Переваги:

- Приховує складність підсистеми.
- Спрощує інтеграцію з фреймворками чи бібліотеками.

Недоліки:

- Може стати великим монолітним об'єктом, прив'язаним до багатьох компонентів.

Шаблон «Bridge» (Міст):

Структура:

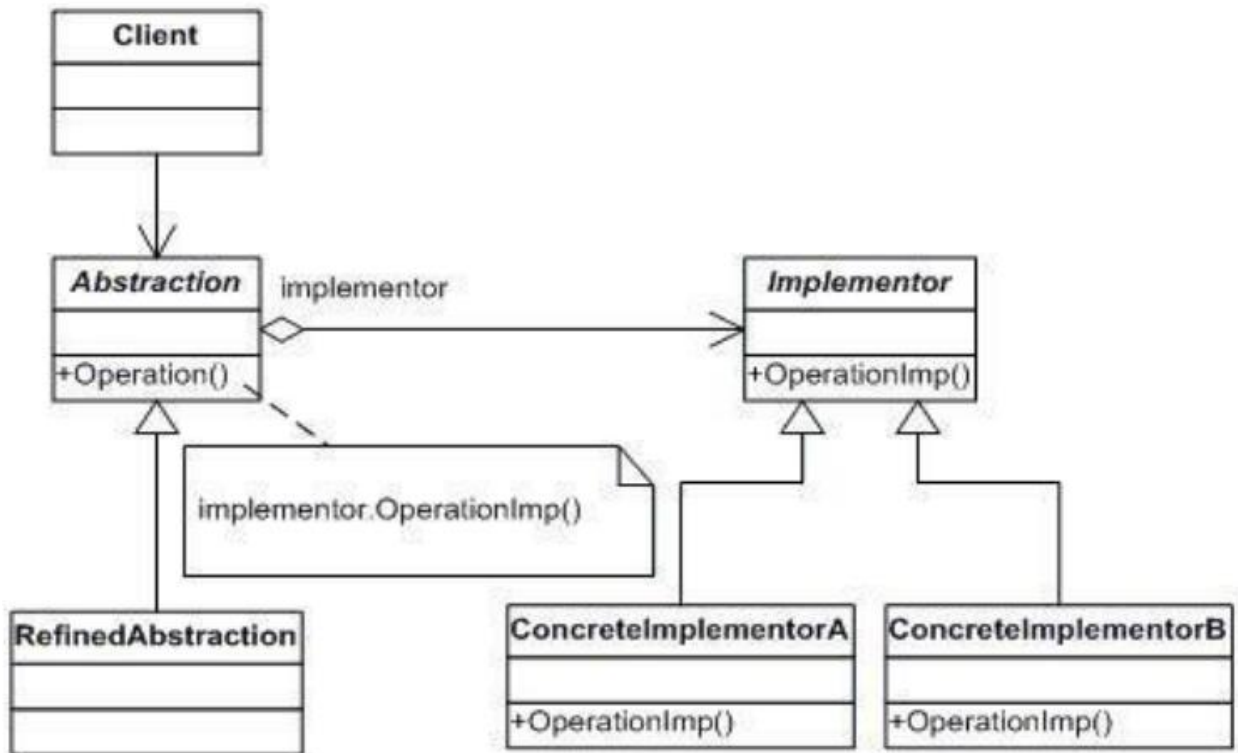


Рис. 1 – Структура шаблону Bridge

Призначення:

Bridge дозволяє розділити абстракцію і її реалізацію, забезпечуючи їхню незалежність і гнучкість. Це спрощує розширення як абстракцій, так і реалізацій.

Проблема:

Коли потрібно поєднати різні аспекти (наприклад, колір і форму фігур), створення окремих класів для кожної комбінації призводить до надмірної кількості класів. Наприклад, додавання нової фігури чи кольору вимагає створення численних нових підкласів.

Рішення:

Шаблон Bridge пропонує замінити спадкування композицією. Абстракція (наприклад, фігура) зберігає посилання на реалізацію (наприклад, колір), дозволяючи їм розвиватися незалежно.

Приклад:

Два міста, з'єднані мостом. Міст виступає посередником між ними, дозволяючи взаємодію без необхідності зміни самих міст.

Переваги:

- Підтримує платформно-незалежність програм.
- Реалізує принцип відкритості/закритості.
- Зменшує кількість необхідних класів.

Недоліки:

- Ускладнює код через додаткові класи і зв'язки.

Шаблон «Template Method»:**Призначення:**

Шаблонний метод дозволяє визначити загальний алгоритм в базовому класі, залишаючи конкретну реалізацію його кроків для підкласів.

Проблема:

Різні алгоритми часто мають спільну структуру, але повторюють загальні частини в коді, що ускладнює його підтримку.

Рішення:

Винести спільні кроки алгоритму в базовий клас і визначити абстрактні методи для кроків, що відрізняються.

Приклад:

Будівництво типових будинків: фундамент, стіни, дах — спільні етапи, але конкретні матеріали чи техніки можуть відрізнятися.

Переваги:

- Полегшує повторне використання коду.
- Забезпечує єдину структуру алгоритмів.

Недоліки:

- Обмежує гнучкість алгоритму через фіксовану структуру.
- Збільшення кроків ускладнює підтримку шаблону.

Реалізувати не менше 3-х класів відповідно до обраної теми

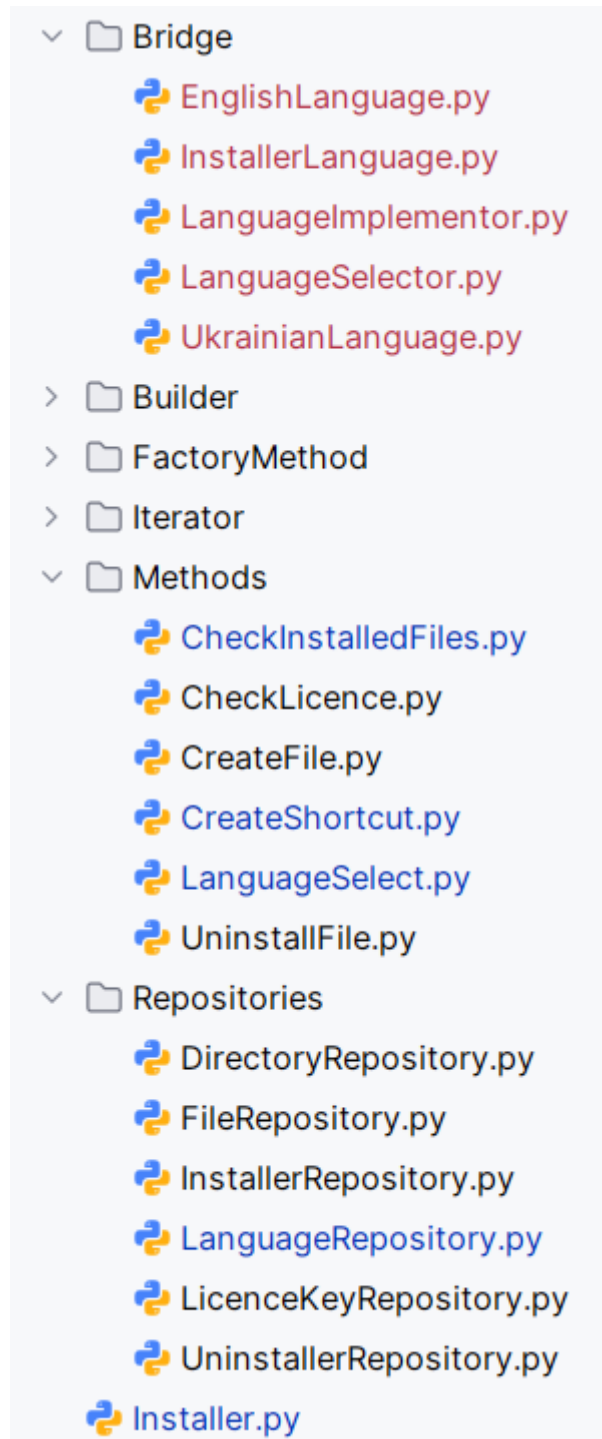


Рис. 2 – Структура проекту

В цій лабораторній роботі було реалізовано частину генератора інсталяційних пакетів з використанням шаблону bridge, а саме вибір мови інсталятора.

Реалізувати один з розглянутих шаблонів за обраною темою
Згідно з завданням, було використано та реалізовано шаблон bridge.

1. LanguageRepository:

```
class LanguageRepository: 2 usages  ± FryMo
    def __init__(self):  ± FryMondo
        self.id = None
        self.language_eng = "English"
        self.language_ukr = "Українська"

    def getEnglish(self): 1 usage (1 dynamic)
        return self.language_eng

    def getUkrainian(self):  ± FryMondo
        return self.language_ukr
```

Рис. 3 – Клас LanguageRepository

- Відповідає за зберігання та надання доступу до даних мов.
- Містить два поля: language_eng (English) і language_ukr (Українська), які визначають доступні мови.

2. LanguageImplementor (Implementor):

```
# Implementor
class LanguageImplementor: 8 usa
    def get_message(self, key):
        pass
```

Рис. 4 – Клас LanguageImplementor

- Абстрактний клас, який визначає базовий інтерфейс для отримання повідомлень мовою, що передбачає метод get_message(key).

3. LanguageSelector (Abstraction):

```
# Abstraction
class LanguageSelector: 2 usages  ± FryMondo
    def __init__(self, implementor: LanguageImplementor):
        self.implementor = implementor

    def get_message(self, key, **kwargs): 7 usages (7 dynamic)
        message = self.implementor.get_message(key)
        return message.format(**kwargs)
```

Рис. 5 – Клас LanguageSelector

- Забезпечує абстрактний інтерфейс для роботи з мовними реалізаціями через об'єкт типу `LanguageImplementor`.
- `get_message(key, **kwargs)`: Отримує повідомлення на обраній мові та форматує його з переданими параметрами.

4. `EnglishLanguage (ConcreteImplementorA)`:

```
class EnglishLanguage(LanguageImplementor): 2 usages  ± FryMondo
    messages = {
        "checking_files": "Checking installed files...",
        "files_checked": "All files checked.",
        "creating_shortcut": "Creating shortcut...",
        "shortcut_created": "Shortcut '{name}' created in '{path}'",
        "enter_license_key": "Enter the licence key: ",
        "license_key_valid": "Licence key is valid. Installation continues...",
        "license_key_invalid": "Invalid licence key. Please try again."
    }

    def get_message(self, key): 7 usages (7 dynamic)  ± FryMondo
        return self.messages.get(key, "Unknown message")
```

Рис. 6 – Клас `EnglishLanguage`

- Конкретна реалізація `LanguageImplementor`, яка забезпечує повідомлення англійською мовою.
- Поле `messages` зберігає ключі та відповідні повідомлення.
- Метод `get_message(key)` повертає текстове повідомлення англійською за заданим ключем або "Unknown message", якщо ключ не знайдено.

5. `UkrainianLanguage (ConcreteImplementorB)`:

```
class UkrainianLanguage(LanguageImplementor): 2 usages  ± FryMondo
    messages = {
        "checking_files": "Перевірка встановлених файлів...",
        "files_checked": "Всі файли перевірено",
        "creating_shortcut": "Створення ярлика...",
        "shortcut_created": "Ярлик '{name}' створено в '{path}'",
        "enter_license_key": "Введіть ліцензійний ключ: ",
        "license_key_valid": "Ліцензійний ключ вірний. Продовження інсталяції...",
        "license_key_invalid": "Невірний ліцензійний ключ. Спробуйте ще раз."
    }

    def get_message(self, key): 7 usages (7 dynamic)  ± FryMondo
        return self.messages.get(key, "Невідоме повідомлення")
```

Рис. 7 – Клас `UkrainianLanguage`

- Конкретна реалізація LanguageImplementor, яка забезпечує повідомлення українською мовою.
- Поле messages містить ключі та відповідні повідомлення українською.
- Метод get_message(key) повертає текстове повідомлення українською за заданим ключем або "Невідоме повідомлення", якщо ключ не знайдено.

6. InstallerLanguage (RefinedAbstraction):

```
class InstallerLanguage(LanguageSelector): 2 usages  ± FryM
    def __init__(self, implementor: LanguageImplementor):
        super().__init__(implementor)
```

Рис. 8 – Клас InstallerLanguage

- Розширює функціональність класу LanguageSelector, дозволяючи використовувати його в контексті інстлятора.

7. languageSelect():

```
# Метод для вибору мови інстлятора
def languageSelect(language): 2 usages  ± Fr
    implementor = EnglishLanguage()

    if language == "Українська":
        implementor = UkrainianLanguage()

    return InstallerLanguage(implementor)

def set_language(language): 2 usages  ± FryM
    global selected_language
    selected_language = language

def get_language(): 11 usages  ± FryMondo
    global selected_language
    return selected_language
```

Рис. 9 – Метод languageSelect()

- Приймає мову як параметр (English або Українська) і повертає об'єкт InstallerLanguage з відповідною реалізацією (EnglishLanguage або UkrainianLanguage).

8. Installer:

```
class Installer: 1 usage 2 FryMondo *  
    def language_select(self, language): 1 usag  
        set_language(languageSelect(language))
```

Рис. 10 – Клас Installer

- Використовує функцію languageSelect() для вибору мови, забезпечуючи доступ до локалізованих повідомлень у процесі роботи інсталятора.

Результат:

```
Selected language: English  
Creating shortcut...  
Shortcut 'File' created in 'C:/Users/Username/Desktop'  
Checking installed files...  
{'id': 1, 'file_name': 'readme', 'file_path': 'C:/', 'file_type': '.txt'}  
{'id': 2, 'file_name': 'installer', 'file_path': 'C:/', 'file_type': '.exe'}  
All files checked.
```

Рис. 11 – Вибір англійської мови

```
Selected language: Українська  
Створення ярлика...  
Ярлик 'File' створено в 'C:/Users/Username/Desktop'  
Перевірка встановлених файлів...  
{'id': 1, 'file_name': 'readme', 'file_path': 'C:/', 'file_type': '.txt'}  
{'id': 2, 'file_name': 'installer', 'file_path': 'C:/', 'file_type': '.exe'}  
Всі файли перевірено
```

Рис. 12 – Вибір української мови

Діаграма класів для шаблону bridge

Діаграма класів для шаблону bridge:

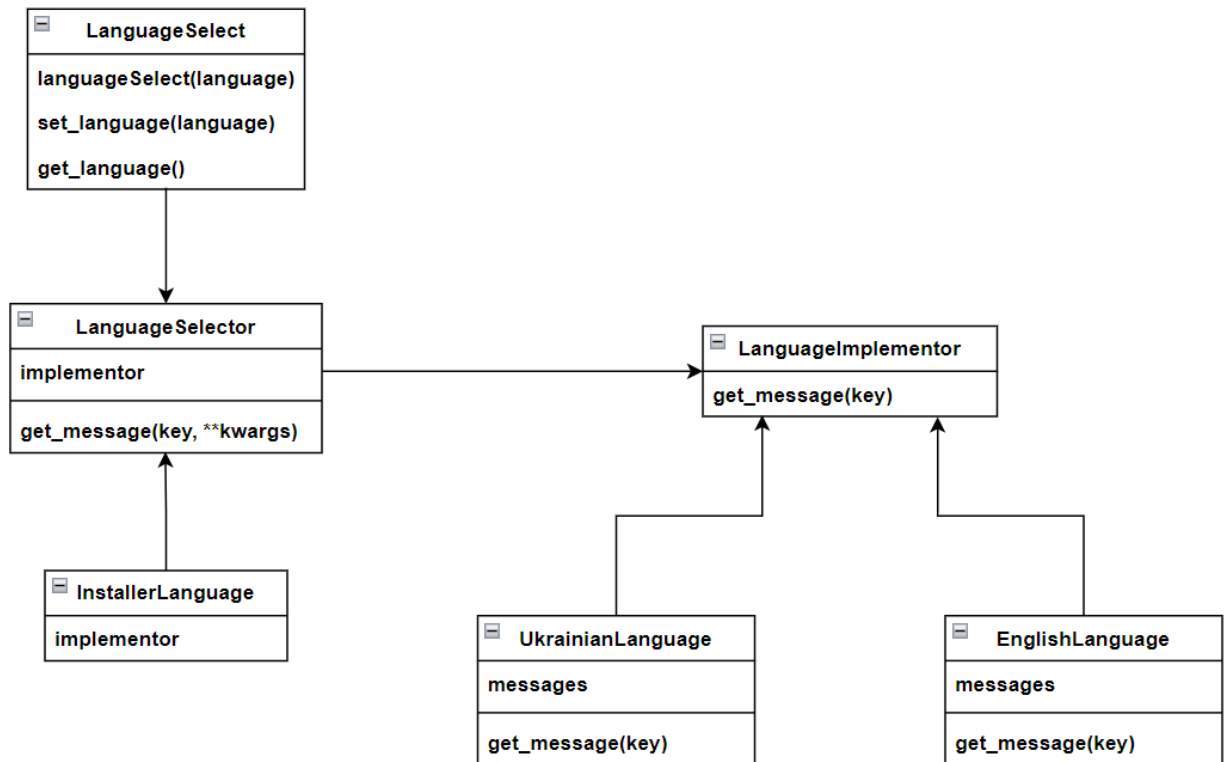


Рис. 13 – Діаграма класів для шаблону bridge

Проблема, яку допоміг вирішити шаблон bridge

Використання шаблону Bridge у проєкті "Installer Generator" вирішило проблему розділення абстракції (вибір і використання мови) та її реалізації (конкретні мовні повідомлення).

Завдяки шаблону Bridge вдалося:

- Розділити логіку вибору мови та локалізації: Абстракція у вигляді **LanguageSelector** забезпечує єдиний інтерфейс для отримання повідомлень, тоді як реалізація мов (**EnglishLanguage**, **UkrainianLanguage**) є незалежною.
- Зменшити залежність між компонентами: Код інстлятора не залежить від конкретних мов, що спрощує його підтримку та адаптацію.
- Забезпечити гнучкість у додаванні нових мов: Для підтримки нової мови достатньо створити новий клас, що реалізує **LanguageImplementor**, без змін у базовому коді.
- Інкапсулювати локалізовані повідомлення: Усі повідомлення згруповані в окремих реалізаціях **LanguageImplementor**, що спрощує їх управління.

- **Покращити масштабованість:** Додавання нових мов або змін у повідомленнях не вимагає модифікації класів, що використовують локалізацію.

Це рішення зробило процес управління локалізацією модульним, гнучким і зручним для розширення, забезпечивши підтримку кількох мов в інсталяторі без ускладнення основного коду.

Переваги використання шаблону bridge

Шаблон Bridge дозволив створити гнучку архітектуру для підтримки локалізації в проекті "Installer Generator", розділивши абстракцію (вибір мови) та реалізацію (локалізація повідомлень). Це забезпечило такі переваги:

1. Розділення обов'язків (SRP):

- Клас LanguageSelector відповідає лише за вибір та форматування повідомлень, залишаючи реалізацію мов у класах EnglishLanguage та UkrainianLanguage.
- Реалізації мов інкапсулюють повідомлення, що дозволяє легко змінювати або додавати нові мови без впливу на абстракцію.

2. Гнучкість:

- Нові мови можуть бути додані, реалізуючи інтерфейс LanguageImplementor. Наприклад, для французької мови достатньо створити клас FrenchLanguage, не змінюючи основну логіку вибору мови.
- Повідомлення різними мовами зберігаються окремо, що забезпечує легкість у їхньому оновленні.

3. Зменшення залежності компонентів:

- Інсталятор (Installer) не залежить від конкретних мов, а взаємодіє лише з об'єктами типу LanguageSelector.
- Це дозволяє використовувати один інтерфейс для роботи з усіма реалізаціями мов.

4. Масштабованість:

- Легко адаптується до змін вимог — наприклад, підтримки нових мов чи форматів повідомлень.

- Можливість розширення без зміни базового коду, що відповідає принципу відкритості/закритості (ОСР).

5. Читабельність і модульність:

- Повідомлення для кожної мови зберігаються у відповідному класі (EnglishLanguage, UkrainianLanguage), що спрощує їхнє управління.
- Клієнтський код залишається чистим і зрозумілим, оскільки вибір мови інкапсулюється в одному місці.

Код та висновок

Код можна знайти за посиланням:

<https://github.com/FryMondo/TRPZ/tree/master/InstallGenerator>

Висновок: У цій лабораторній роботі реалізовано шаблон Bridge для генератора інсталяційних пакетів, що забезпечило гнучкість і розширюваність системи. Реалізація включала класи, які розділяють абстракцію вибору мови та її реалізацію, зокрема LanguageSelector, EnglishLanguage, UkrainianLanguage. Використання шаблону дозволило інкапсулювати логіку локалізації, зменшити залежності між компонентами, забезпечити масштабованість системи та полегшити підтримку коду. Це рішення зробило систему модульною, зручною для розширення та адаптованою до нових вимог.